NATIONAL TEACHERS COLLEGE

School of Arts, Sciences and Technology | Department of Computer Studies

---

CC105 – DATA STRUCTURES W/LAB

DYNAMIC DATA ENGINE & BENCHMARKING SUITE

University Student Service & Queue System

Terminal Assessment 2 / Milestone 2
Section 2.4 BSIT | October 19, 2026

GROUP PROJECT PRESENTATION

01. PROJECT OVERVIEW

Problem, intended users, and proposed system

- Universities process assessment, clearance, and transcript requests. Manual record handling can cause disorganized information, slow retrieval, and inefficient queues.

- Proposed solution: a menu-driven C++ terminal application that manages student records and organizes regular and priority service requests.

- Intended users: university administrative personnel, student-services staff, and authorized operators.

02. PROJECT OBJECTIVES

General and specific objectives

- General objective: develop a C++ system demonstrating dynamic data structures, searching and sorting, memory management, and algorithm benchmarking.

- Implement student-record operations using a dynamically allocated singly linked list.

- Use a linked-list stack for undo history and linked-list queues for regular and priority service.

- Compare Linear Search, Binary Search, Bubble Sort, Selection Sort, and Insertion Sort using operation counts and runtime.

03. MAJOR SYSTEM FUNCTIONS

Requirements and data structures

System function	Description	Data structure
Student records	Add, display, search, delete, and update	Singly linked list
Undo / audit	Store deleted records and previous GPA values	Linked-list stack
Service queues	Enqueue, inspect, and serve regular/priority requests	Linked-list queues
Search / sort	Search student IDs and sort records or IDs	Dynamic array
Benchmarking	Count operations and measure execution time	Arrays and metrics

04. DATA STRUCTURE DESIGN

Structure selection and expected complexity

Feature	Selected structure	Reason / expected complexity
Student registry	Singly linked list	Dynamic storage; common operations O(n)
Undo history	Linked-list stack	LIFO; push/pop O(1)
Regular service	Linked-list queue	FIFO; enqueue/dequeue O(1)
Priority service	Sorted linked list	Priority insertion O(n); dequeue O(1)
Search/sort workspace	Dynamic array	Index-based algorithms; sorts generally $O(n^2)$

05. STUDENT REGISTRY

Singly linked list implementation

- Each node contains a Student value and a Node* next pointer. The head identifies the first node; the final node points to NULL.

- Operations include add, remove, find, update, display, and count.

- The worksheet documents O(n) traversal for registry operations; the implementation does not use a tail pointer.

06. STACK AND QUEUES

Undo history and service processing

- Undo stack follows LIFO: the newest action is handled first. Push and pop are O(1); displaying history is O(n).

- Regular service queue follows FIFO. Enqueue, dequeue, and peek are O(1); display is O(n).

- Priority queue inserts according to priority in O(n), while dequeue and peek are O(1). Equal-priority students retain arrival order.

07. SEARCHING ALGORITHMS

Linear Search and Binary Search

Criteria	Linear Search	Binary Search
Best case	O(1)	O(1)
Average case	O(n)	O(log n)
Worst case	O(n)	O(log n)
Sorted data required?	No	Yes
Worksheet note	May inspect records one by one	Halves search interval each step

Binary Search requires sorted data to correctly choose which half to continue searching.

08. SORTING ALGORITHMS

Bubble Sort, Selection Sort, and Insertion Sort

- Bubble Sort compares neighboring elements and swaps out-of-order values.
- Selection Sort locates the next minimum element and places it in position.
- Insertion Sort shifts larger preceding values and inserts the current key.
- The worksheet lists $O(n^2)$ worst-case complexity for all three sorting algorithms.

09. BENCHMARK RESULTS

Recorded worksheet metrics for N = 100

Algorithm (N=100)	Comparisons	Swaps / shifts	Recorded time
Linear Search	100	0	0.000 ms
Binary Search	11	0	0.000 ms
Bubble Sort	4,859	2,538	0.000 ms
Selection Sort	4,950	97	0.000 ms
Insertion Sort	2,630	2,538	0.000 ms

Note: The worksheet's 0.000 ms readings do not support a reliable fastest-to-slowest ranking.

10. EMPIRICAL ANALYSIS

Interpreting the recorded results

- The worksheet reports execution times of 0.000 ms, which are too coarse to reliably determine the fastest algorithm.

- Operation counts provide a useful comparison, but measurements depend on dataset size, input arrangement, hardware, compiler settings, and timing precision.

- Binary Search is expected to reduce the search interval by half, provided the data is sorted.

11. DYNAMIC MEMORY MANAGEMENT

Allocation, deallocation, and audit

- Linked-list nodes and dynamic arrays are allocated with new / new[].

- Removed nodes are released with delete; arrays are released with delete[].

- Registry, stack, and queue cleanup routines/destructors are documented. Runtime leak checking is still needed to verify leak-free execution.

12. TESTING AND VALIDATION

Selected worksheet test coverage

Test area	Worksheet outcome
Valid / invalid input	Pass
Empty undo and queue conditions	Pass
Single and multiple student records	Pass
Existing and missing student search	Pass
Record deletion and sorting demo	Pass
Boundary conditions	Pass

Outcomes are transcribed from the supplied worksheet test log.

13. BUGS AND DEBUGGING

Integration issues recorded in the worksheet

- Queue entries may remain after a student is deleted from the registry.
- Undo may restore a registry record without restoring its queue entry; queue data may also retain outdated GPA values.
- Input validation should reject whitespace-only names and handle end-of-file cleanly.
- The worksheet records fixes, but integration behavior should continue to be verified during demonstration.

14. TEAM CONTRIBUTIONS

Assigned modules and reported contribution

Group member	Assigned responsibility	Contribution
Lovely Esther Ang	Linked-list student registry	20%
Jinea Elgin Benjamin	Searching, sorting, instrumentation	20%
Angeline Francisco	Stack, queue, priority queue	20%
Brittanny Octaviano	CLI and performance analysis	20%
Don Gabriel Poblete	Benchmarking suite	20%
Total		100%

15. LIMITATIONS AND NEXT STEPS

Proposed improvements

- Strengthen synchronization among the registry, undo history, and service queues.
- Improve benchmark timing precision and test larger datasets and different input arrangements.
- Expand input validation, run memory-leak checks, and continue edge-case testing.
- Improve the menu interface and verify all modules together before the practical defense.

16. CONCLUSION

Project summary

- The system applies dynamic data structures and fundamental algorithms to a university student-service workflow.

- The worksheet documents implemented features and test cases, while also identifying performance-measurement and data-synchronization areas that need further verification.

Source: CC105 Terminal Assessment 2 / Milestone 2 Student Worksheet (provided).